

ProtoBuf协议 Android 蓝牙 SDK 操作说明

接入说明

1. 请将 SDK 中的 **xxx.aar** 文件拷贝到自己的 project 工程中的 lib 下。
2. 在自己 project 的 app 中的 build.gradle 的 dependencies 下面添加以下代码：

```
implementation(name: 'blesdk240531', ext: 'aar')
implementation 'com.google.protobuf:protobuf-java:3.11.0'
implementation 'io.reactivex.rxjava2:rxandroid:2.1.0'
```

然后在 build.gradle 的 android 部分添加如下代码：

```
repositories {
    flatDir {
        dirs 'libs'
    }
}
```

3. 在 Application 中启动 **BleService.class**，并添加以下代码：

```
SuperBleSDK.addBleListener(this, new IDataReceiveHandler());
```

具体说明参考示例代码的 **BleApplication**。

4. 注册一个广播接收器 MyReceiver.class 继承 BluetoothCallbackReceiver.class，这样全局都可以接收到该接收器的数据，可统一处理分发给需要数据的页面。拷贝 Demo 中 BaseActionUtils.class 和 BluetoothCallbackReceiver.class 到自己的工程中，可以写在自己 Application 的 onCreate()，具体代码如下：

```
IntentFilter intentFilter = BaseActionUtils.getIntentFilter();
MyReceiver receiver = new MyReceiver();
LocalBroadcastManager.getInstance(this).registerReceiver(receiver,
    intentFilter);
```

5. 复制 Demo 中 BleApplication.class 中的代码 SuperBleSDK.addBleListener(this, new IDataReceiveHandler()) 包括实现和发送的广播到代码到自己的 Application 中。
6. 初始化 SDKType：调用代码：

```
SuperBleSDK.switchSDKType(this, Constants.Bluetooth.Zeroner_protobuf_Sdk);
```

注意第二个参数选择 **Bluetooth.Zeroner_protobuf_Sdk**，然后调用：

```
MyApplication.getInstance().getmService().setSDKType(this.getApplicationContext()  
, Constants.Bluetooth.Zeroner_protobuf_Sdk);
```

注意此方法必须在 **BleService.class** 启动才能执行，如果启动项目就开始扫描建议加一个延时。如下：

```
Handler handler = new Handler(Looper.getMainLooper());  
handler.postDelayed(new Runnable() {  
    @Override  
    public void run() {  
  
        MyApplication.getInstance().getmService().setSDKType(this.getApplicationContext()  
( ), Constants.Bluetooth.Zeroner_protobuf_Sdk);  
    }  
}, 3000);
```

扫描连接

```
//=====如果你的项目已经适配android 12以上，扫描之前需要先动态申请蓝牙扫描权限  
//===== 如果还没有适配android 12，扫描之前则需要申请 定位权限  
// --- android 12以上 在 AndroidManifest.xml 添加  
    <uses-permission android:name="android.permission.BLUETOOTH" />  
    <uses-permission android:name="android.permission.BLUETOOTH_SCAN" />  
    <uses-permission android:name="android.permission.BLUETOOTH_ADMIN" />  
    <uses-permission android:name="android.permission.BLUETOOTH_CONNECT" />  
// --- android 12以下 在 AndroidManifest.xml 添加  
    <uses-permission android:name="android.permission.BLUETOOTH" />  
    <uses-permission android:name="android.permission.BLUETOOTH_ADMIN" />  
    <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />  
    <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />  
//然后动态申请权限相应的权限  
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) {  
        ActivityCompat.requestPermissions(this,  
            new String[]{  
Manifest.permission.BLUETOOTH_SCAN,Manifest.permission.BLUETOOTH_CONNECT },  
            101);  
    }else{  
        ActivityCompat.requestPermissions(this,
```

```

        new String[]{ Manifest.permission.ACCESS_FINE_LOCATION },
        101);
    }

    /**
     * 没有以上的权限 扫描蓝牙设备时 会出现 扫描不到设备的现象, 或者闪退的现象
     * 没有以上的权限 扫描蓝牙设备时 会出现 扫描不到设备的现象, 或者闪退的现象
     */

```

1. 建议使用 `BluetoothUtil.class` 来进行操作。扫描调用 `BluetoothUtil.startScan()` 方法。通过 `LocalBroadcastManager.getInstance(context).registerReceiver(你的接收器对象, BaseActionUtils.getIntentFilter())` 来回调。有三个回调方法如下解释：
 - `onPreConnect()` 发起连接回调
 - `onScanResult(WristBand band)` 扫描结果回调，在此回调方法中返回扫描的手环
 - `connectStatue(boolean isConnected)` 连接状态回调，断开和连接成功时候此方法
 - `onBluetoothInit()` 连接成功，可以和蓝牙进行数据交互，收发指令调用此方法。一般情况在此方法处理蓝牙连接成功的标识，以便可以进行发送接收指令
2. 调用 `BluetoothUtil.stopScan()`；来停止扫描，调用 `BluetoothUtil.connect(band)` 来进行连接。注意 **band 对象是 `onScanResult(WristBand band)` 中的 band。**

收发指令

1. 发送指令需要在 `onBluetoothInit()` 之后进行发送数据；
2. 有返回值的命令需要在最后加 `BackgroundThreadManager.getInstance().addWriteData(context, cmd)` 如：

```

/* 设置时间 */
byte[] bytes = SuperBleSDK.getSDKSendBluetoothCmdImpl(context).setTime();
BackgroundThreadManager.getInstance().addWriteData(context, bytes);

```

APP下发类型指令

一般下发指令手环返回model解析一样的,通知成功,发送成功或者失败.

- 设置时间

```
// 设置时间 当前系统的实时时间
public byte[] setTime()
// 设置时间带参数 ,单位(s)
byte[] setTime(long time)
```

▪ 设置心率报警

```
/**
 * 设置心率报警
 *
 * @param enable    使能位
 * @param hrH       心率最大值 // 50~200
 * @param hrL       心率最小值 // 40~190
 * @param second    心率持续时间 (s)
 * @param interval  心率报警间隔时间(min)
 */
byte[] setHeartAlarm(boolean enable, int hrH, int hrL, int second, int interval)
```

▪ 设置用户信息

```
/**
 * 设置用户信息
 *
 * height 高 默认170
 * weight 体重 kg 默认60
 * gender 性别 false 男 true 女
 * age    年龄 默认20
 * walk灵敏度 50-200之间 默认100(请勿轻易修改)
 * run 灵敏度 50-200之间 默认100(请勿轻易修改)
 * wristCircumference 手腕围度 80-230,单位 毫米
 * hasHypertension 是否有高血压历史 false-无,true-有
 */

byte[] setUserConf(int height, int weight, boolean gender, int age)
byte[] setUserConf(int height, int weight, boolean gender, int age, int walk, int run)
byte[] setUserConf(int height, int weight, boolean gender, int age, int walk, int run, int wristCircumference, boolean hasHypertension)
```

示例:

```
byte[] bytes = ProtoBufSendBluetoothCmdImpl.getInstance().setUserConf( xxx );
BackgroundThreadManager.getInstance().addWriteData(context,bytes);
```

▪ 设置sn号

```
/**
 * snStr ，字符串，最大30个字符串，不含中文
 */
byte[] setSn(@NonNull String snStr)
    示例：
        byte[] bytes = ProtoBufSendBluetoothCmdImpl.getInstance().setSn( xxx );
        BackgroundThreadManager.getInstance().addWriteData(context,bytes);
```

▪ 设置运动目标

```
/**
 * 设置全局配置
 *
 * @param calorie 卡路里 default = 400 kC
 * @param step    default = 10000 unit seconds
 * @param distance default = 10000 unit meters
 */
byte[] setGoalConf(int calorie, int step, int distance)
```

▪ 设置GPS经纬度海拔

```
/**
 * GPS
 *
 * @param altitude 海拔
 * @param latitude 纬度
 * @param longitude 经度
 */
byte[] setGnssConf(int altitude, String latitude, String longitude)
```

▪ 设置血压

```

/**
 * 血压
 *
 * @param src_sbp//手环测量收缩压
 * @param src_dbp//手环测量舒张压
 * @param dst_sbp//血压计测量收缩压
 * @param dst_dbp//血压计测量舒张压
 * @param dif_sbp//保留 传0
 * @param dif_dbp//保留 传0
 */
byte[] setBpCaliConf(int src_sbp,int src_dbp,int dst_sbp,int dst_dbp,int
dif_sbp,int dif_dbp)

```

■ 设置消息免打扰

```

/**
 * 消息免打扰
 *
 * @param
 * @param policy    true 开启 false 关闭
 * @param startHour 开始小时
 * @param endHour   结束小时
 * @param startMin  结束分钟
 * @param endMin    结束分钟
 */
byte[] setMsgNotificationTime(boolean policy, int startHour, int endHour, int
startMin, int endMin)

```

■ 设置消息通知

```

/**
 * 消息通知
 *
 * @param
 * @param id        保证唯一性.用于拒接传Id
 * @param type      ADDED = 0; // incoming call and sms
 *                  REMOVED = 1; // incoming call accepted or rejected
 *                  UPDATED = 2; // incoming call turn to missed-call
 * @param accept    接收
 * @param reject    拒接
 * @param mute      无声
 * @param title     发送电话称呼 在android上面传""

```

```

* @param detail 发送内容,如果是电话就是电话号码 如果有名称就显示名称
*
*         <p>
*         B.对于来电
*         当有来电时, Peer应向设备发送一个MsgNotify
*         {id = xxx, type = TYPE_CALL, status = ADDED, option = {accept =
true, reject = true, mute = true}, title =“Incoming Call”, detail =“ (0755) 10010”}
*         <p>
*         如果被拒绝或接受, Peer将向设备发送MsgNotify
*         {id = xxx, type = TYPE_CALL, status = REMOVED}
*         <p>
*         否则会错过, Peer将发送一个MsgNotify 给设备
*         {id = xxx, type = TYPE_CALL, status = UPDATED}
*         <p>
*         C.对于短信
*         当SMS出现时, Peer将向设备发送MsgNotify
*         {id = xxy, type = TYPE_SMS, status = ADDED}
*/

public byte[] setMsgNotificationNotifyBySms(int id, int type, boolean accept,
boolean reject, boolean mute, String title, String detail)

```

■ 设置电话通知

```

/**
* 同消息通知
*/

public byte[] setMsgNotificationNotifyByCall(int id, int type, boolean accept,
boolean reject, boolean mute, String title, String detail)

```

■ 设置天气

```

/**
* @param timeMills 单位s
* 注意:如果weatherType设置 EACH_DAY 必须使用当天的凌晨时间作为ID.
*         如果设置 EACH_HOUR 以每小时整点的时间作为ID
* 如:设置 2019-7-15 EACH_DAY 则需要 2019-7-15 00:00:00的时间戳(s)作为ID
*         设置 2019-7-15 08:xx:xx EACH_HOUR 则需要 2019-7-15 08:00:00的时间戳(s)作为ID
*
* @param weatherDesc // required for ADD
*
*         WeatherDesc {
*
*         Sunny = 0; 晴天
*
*         Cloudy = 1; 多云
*
*         Overcast = 2; 阴天
*
*         LightRain = 3;小雨

```

```

*           ModerateRain = 4;中雨
*           HeavyRain = 5;大雨
*           shower = 6;阵雨
*           Snow = 7;雪
*           Haze = 8;雾霾
*           SandStorm = 9;沙尘暴
*           CloudyTurnToFine = 10;多云转晴
*           Thunderstorm = 11;雷雨
*           }
* @param weatherType EACH_HOUR = 0;
*           EACH_DAY = 1;
* @param degreeMax    最高温度
* @param degreeMin    最低温度
* @param pm2p5        pm2.5
*/
public byte[] setWeather(int timeMills, int weatherDesc, int weatherType, int
degreeMax, int degreeMin, int pm2p5)

/**
 * WeatherEvent 同上面的参数.传24小时天气.这样设置每小时都会变化.
 * 注意: timeMills 必须以每小时的ID 比如:2019年7月1日16点30分30秒,
 * 则传2019年7月1日16点0分0秒的时间戳(单位s)作为唯一ID
 *
 */
byte[] setWeather(List<WeatherEvent> weatherEvents)
//清除天气指令
byte[] clearWeather()

```

■ 添加闹钟

```

/**
 * 添加闹钟
 *
 * @param id      AlarmId
 * @param repeat  repeat times
 * @param week    0000 0001    sun 0x01
 *                0000 0010    sat 0x20
 *                0000 0100    ...
 *                0000 1000
 *                0001 0000
 *                0010 0000
 *                0100 0000
 *
 * <p>

```



```

*          sample set mon,tue,wed -- 0111 0000
* @param hour    set hour
* @param min     set min
* @param text    set text 最长不要超过12字符
*/
byte[] addAlarm(int id, boolean repeat, int week, int hour, int min, String text)
//根据ID移除闹钟
byte[] removeAlarm(int id)
//清除闹钟
byte[] clearAlarm()

```

■ 设置久坐提醒

```

/**
 * 久坐提醒
 *
 * @param repeat
 * @param week      同setAlarm()
 * @param startHour 开始时间 // 0~23小时
 * @param endHour   结束时间   // 1~24小时
 * @param duration  // 0~1440 //久坐提醒的间隔 单位(分钟)
 * @param threshold // duration设置的时间多少步内
 *
 *                  比如:duration设置60分钟 threshold设置50
 *
 *                  那就是60分钟步数为50步内为触发久坐提醒
 */
byte[] setSedentariness(boolean repeat, int hash, int week, int startHour, int
endHour, int duration, int threshold)
//清除久坐提醒
byte[] clearSedentariness()

```

```

/**
 * 久坐提醒
 * 使用此指令设置久坐提醒
 * @param repeat
 * @param week      同setAlarm()
 * @param startHour 开始时间 // 0~23
 * @param endHour   结束时间   // 0~23
 * @param duration  // 设置 x表示 5 * x 分钟 如:1就表示5分钟
 * @param threshold
 * @param isDisturb true 开启免打扰 false 关闭
 */

```

```
public byte[] setSedentariness(boolean repeat, int hash, int week, int startHour, int endHour, int duration, int threshold, boolean isDisturb)
```

▪ 添加日程

```
/**
 *
 * @param id 唯一ID
 * @param timeMillis 单位s
 * @param text 日程信息
 * @return
 */
byte[] addCalendar(int id, int timeMillis, String text)

//移除日程
public byte[] removeCalendar(int hash, int second)
```

▪ 设置语言

```
/**
 * 设置语言
 *
 * @param context
 * @param language English = 0;
 *                  Chinese = 1;
 *                  Italian = 2;
 *                  Japanese = 3;
 *                  France = 4;
 *                  German = 5;
 *                  Portuguese = 6;
 *                  Spanish = 7;
 *                  Russian = 8;
 *                  Korean = 9;
 *                  Arabic = 10;
 *                  Vietnam = 11;
 *                  Polish = 12;
 */
void setLanguage(Context context, int language)
```

▪ 设置距离,温度,时间,日期格式

```
/**
 * 设置距离单位
 *
 * @param distance_unit false[default]: metric unit, true 1: imperial units
 */
byte[] setDistanceUnit(boolean distance_unit)
```

```
/**
 * 设置温度单位
 *
 * @param temperature_unit false[default]: Celsius, true: Fahrenheit
 */

byte[] setTemperatureUnit(boolean temperature_unit)
```

```
/**
 * 设置小时格式
 *
 * @param hour_format false[default]: 24hour, true1: 12hour
 */
byte[] setHourFormat(boolean hour_format)
```

```
/**
 * 设置日期格式
 *
 * @param date_format 0[default]: month/day, 1: day/month
 */
byte[] setDateFormat(boolean date_format)
```

■ 设置自动心率

```
/**
 * 设置自动心率
 *
 * @param autoHeartrate true on false off
 */
byte[] setAutoHeartrate(boolean autoHeartrate)
```

- 设置佩戴方式（左右手）

```
/**
 * 设置左右手
 * @param habitual_hand , false-left hand
 */
byte[] setHabitualHand(boolean habitual_hand)
```

- 设置自动运动

```
/**
 * 设置自动运动
 *
 * @param auto_sport true on false off
 */
byte[] setAutoSport(boolean auto_sport)
```

- 设置翻腕亮屏

```
/**
 * @param lcdGsswitch 0 disable lcd switching by gesture, 1[default]: enable lcd
switching by gesture
 * @param startHour 0[default]~23: the start hour of switching lcd by gesture
 */
byte[] setLcdGsTime(boolean lcdGsswitch, int startHour, int endHour)
```

- 设置电机震动

```
/**
 * 电机震动类型
 *
 * @param mode 震动模式
 * mode = 4 断奏
           5 波浪
           7 心跳
           8 放射
          11 灯塔
          12 交响乐
          15 快速
```

```

*
* @param round 震动次数 最好不要超过15
* @param type  ALARM_CLOCK = 0;
*               INCOMING_CALL = 1;
*               SMS = 2;
*               SEDENTARINESS = 3;
*               CHARGING = 4;
*               CALENDAR = 5;
*               DISTANCE_ALARM=6;
*               HEARTRATE_ALARM=7;
*               OTHERS = 8;
*/
byte[] setMototConf(int mode, int round, int type)

/**
 * 设置电机震动
 *
 * @param mode 同setMototConf
 * @param round
 */
byte[] setMotorVibrate(int mode, int round)

```

■ 设置智拍

```

/**
 * 设置智拍模式
 *
 * @param mode RT_MODE_BACK_NORMAL = 0; //退出
 *             RT_MODE_ENTER_CAMERA = 1; 进入智拍
 */
byte[] setSmartShotData(int mode)

```

■ 一键测量健康开关

```

/**
 * 一键测量健康开关
 * 开启后，设备会在60s内进行sensor数据测量，测量完成返回1条结果数据
 * 结果内容在[一键测量健康的结果数据]部分阐述
 *
 * @param all_measure true on false off
 */
public byte[] setAllMeasure(boolean all_measure)
示例：
    byte[] bytes =
SuperBleSDK.getSDKSendBluetoothCmdImpl(context).setAllMeasure(true);
    BackgroundThreadManager.getInstance().addWriteData(context,bytes);

```

■ 测量实时心率

```

/**
 * 开始 测量实时心率
 */
SuperBleSDK.getSDKSendBluetoothCmdImpl(context).measureHeartRate(context);

/**
 * 停止 测量实时心率
 */
SuperBleSDK.getSDKSendBluetoothCmdImpl(context).stopMeasurement(context);

/**
 * 接受实时测量的心率
 * 在回调的onDataArrived方法接收dataType 为 0x2a37的数据
 * 0未佩戴、0xFF检测中、其他数字为正常心率值
 */
@Override
public void onDataArrived(Context context, int ble_sdk_type, int dataType, String data) {
    if (dataType == 0x2a37) {
        // 0未佩戴、0xFF检测中、其他数字为正常心率值
        switch (data) {
            case "0": //未佩戴
                break;
            case "255": // 检测中
                break;
            default: //心率值
                break;
        }
    }
}

```

```

        }
    }
}

```

手环上报类型指令

返回值是以json字符串形式返回的,可以根据谷歌Gson或者阿里fastJson解析model

■ 获取设备信息

```

byte[] getHardwareFeatures()

//retrurn model:
public class ProtoBufHardwareInfo {

    private String version;//版本号
    private String model;//名称
    private String mac;
    private String deviceTime;
    private String factory;
    private String fota;
    private int fotaType;
    /**
     * sn码
     */
    private String sn;
}

```

■ 获取电量

```

//获取电量
byte[] getBattery() 当为电量时.isBattery = true

//返回model
public class ProtoBufRealTimeData {
    /**

```

```

    *
    是否显示电量 电量多少 是否在充电
    */
private boolean isBattery;
private int level;//电量
private boolean charging;//是否充电

/**
 * 是否显示健康数据 卡路里 单位(kcal) 距离(0.1m) 步数
 */
private boolean isHearth;
private float calorie;
private int distance;
private int steps;

/**
 * 是否都显示时间 秒
 */
private boolean isTime;
private int seconds;

/**
 * 是否显示按键时间 按键模式 0 normal 1 camera
 */
private boolean isKey;
private int keyMode;

/**
 * 是否显示sensor数据
 */
private boolean isSensorData;
private int sensorType; //1 ECG 2 PPG 4 MAG 8 GYRO 16 ACC
private int sensorSeconds;//时间戳(s)
private int sensorSeq;//seq
private int[] sensorDataList;//具体的数值
}

```

■ 获取健康数据


```
//获取健康数据
byte[] getRealHealthData()

//返回值同获取电量 根据 isHearth 来判断是否显示健康数据,具体内容在电量模块中解释
public class ProtoBufRealTimeData {
    /**
     * 是否显示健康数据 卡路里 距离 步数
     */
    private boolean isHearth;
    private float calorie;
    private int distance;
    private int steps;
}
```

■ 一键测量健康的结果数据

```
// 一键测量健康开关指令下发后, 设备测量一段时间完成后返回
public class ProtoBufRealTimeData {
    /**
     * 一键健康测量结果
     */
    private int fatigue;//疲劳度
    private int stress;//压力值
    private int spo2;//血氧
    private int hr;//心率
    private int sbp;//收缩压
    private int dbp;//舒张压
    private int breathRate;//呼吸率
    private int temperature;//体温
}
```

■ 获取sensor数据

```
/**
 *
 * 获取设备实时sensor数据
 *
 * @param status 0 start 1 pause 2 stop
 *
 * @param type sensor类型 eg:{@link RealtimeData.SensorType#TEMPERATURE_VALUE}
```

```

*          0 NONE
*
*          1 ECG   ... 0000 0001
*          2 PPG   ... 0000 0010
*          4 MAG   ... 0000 0100
*          8 GYRO  ... 0000 1000
*         16 ACC   ... 0001 0000
*
*          .....
*
*          例如:同时发送ECG和PPG  0000 0001 | 0000 0010  =  0000 0011  = 3 以此类推
*
* @return
*/
public byte[] getRtSensorData(int status, int type)

//返回值同获取电量model 根据 isSensorData 来判断是否显示sensor值,具体内容在电量模块中解释
public class ProtoBufRealTimeData {

}

```

■ 开启/关闭实时运动(马拉松)

```

//开启运动
byte[] sportStart =
ProtoBufSendBluetoothCmdImpl.getInstance().setMarathonStatusType(0);
    BackgroundThreadManager.getInstance().addWriteData(Utils.getApp(), sportStart);
//停止运动
byte[] sportStop =
ProtoBufSendBluetoothCmdImpl.getInstance().setMarathonStatusType(2);
    BackgroundThreadManager.getInstance().addWriteData(Utils.getApp(), sportStop);

//以下为接收的实时运动数据(解析0x70数据)
if (dataType == 0x70) {
    ProtoBufRealTimeData realTimeData = JsonTool.fromJson(data,
ProtoBufRealTimeData.class);
    if(realTimeData.getDataType() ==
RealtimeData.RtNotification.DataCase.RT_SPORT_DATA.getNumber()){
        //心率和步频数据
        int reHeart = realTimeData.getSportData().getHeart();
        int stepFrequency = realTimeData.getSportData().getStepFrequency();
        //总步数和总卡路里
        int reStep = realTimeData.getSportData().getSteps();
        float reCalorie = realTimeData.getSportData().getCalories();

    }
}

```

```
}
```

■ 获取设备支持的同步数据

```
/**
 * 获取设备支持的同步数据
 */
byte[] getDataInfo()

//返回model:
public class ProtoBufSupportInfo {

    private boolean support_health;
    private boolean support_gnss;
    private boolean support_ecg;
    private boolean support_ppg;
    private boolean support_rri;
    private boolean support_swim;
}
```

■ 同步历史数据

- 1.先获取index table
- 2.根据index table中的startSeq和endSeq进行历史数据同步。
- 3.同步类型可通过getDataInfo()来获取支持的类型来同步数据,否则会出新发送指令手环不回复的情况导致APP的假死(参考条目'获取设备支持的同步数据')。

```
/**
 *
 * 获取历史数据索引表
 * get index table
 *
 * @param type eg:{@link
com.zeroner.blemidautumn.bluetooth.proto.HisDataOuterClass.HisDataType#TEMPERATURE_D
ATA_VALUE }
 *      HEALTH_DATA = 0;
 *      GNSS_DATA = 1;
 *      ECG_DATA = 2;
 *      .....
 */
```

```

byte[] itHisData(int type)

/**
 * 获取历史数据
 * 根据seq数目，设备会依次返回相应数目的数据
 * 同步过程中不建议多线程操作
 *
 * @param type eg:{@link
com.zeroner.blemidautumn.bluetooth.proto.HisDataOuterClass.HisDataType#TEMPERATURE_D
ATA_VALUE }
        HEALTH_DATA = 0;
 *        GNSS_DATA = 1;
 *        ECG_DATA = 2;
 *        .....
 * @param startSeq 开始seq
 * @param endSeq 结束seq
 */
byte[] startHisData(int type, int startSeq, int endSeq)

```

```

//index table类用于记录同步信息
public class ProtoBuf_index_80 extends DataSupport {

    private long uid;
    private String data_from;
    private int start_idx;start seq
    private int end_idx;end seq
    private int year;
    private int month;
    private int day;
    private int hour;
    private int min;
    private int second;
    private int time;//时间戳
    private int indexType;//类型
}

```

由于同步历史数据不是一条数据，数据量很大，SDK仅提供命令，示例代码中有关于同步数据demo，如下：

```

//同步历史数据
ProtoBufSync.getInstance().syncData();

//返回model
public class ProtoBuf_80_data {

```

```
private long uid;
/**
 * data
 */
private int year;
private int month;
private int day;
private int hour;
private int minute;
private int second;

/**
 * 时间戳(单位s)
 */
private int time;
/**
 * 排序用的
 */
private int seq;
/**
 * 设备名
 */
private String data_from;
/**
 * 睡眠数据
 */
private String sleepData;
private boolean charge;
private boolean shutdown;
/**
 * 健康
 */
private int type;
private int state;
private float calorie;
private int step;
private float distance;
/**
 * 心率
 */
private int min_bpm;
private int max_bpm;
private int avg_bpm;
/**
```

```

        * 疲劳度
        */

private float SDNN;
private float RMSSD;
private float PNN50;
private float MEAN;
private float fatigue;

/**
 * 血压
 *
 */
private int sbp;
private int dbp;
private int bpTime;//bp数据

/**
 * 呼吸训练
 */
private float mdt_SDNN;
private float mdt_RMSSD;
private float mdt_PNN50;
private float mdt_MEAN;
private int mdt_status;
private float mdt_RESULT;
private float mdt_RELAX;
}

```

■ 同步中医数据

```

/**
 * 同步中医数据和 同步历史数据是类似的,
 * 在ProtoBufSync中包含了如何同步中医数据, 步骤
 * 1. 查询手表是否支持中医
 * 2. 获取历史数据索引值, get index table
 * 3. 通过第二步获取到startSeq和endSeq 获取具体的历史数据
 * 4. 同步完成后发送停止同步数据的指令, 目的是让手表不在提醒需要同步数据
 * 同步参考demo中 ProtoBufSync.class 和 ProtoBufDataParsePersenter.class
 * 展示数据参考demo中 YylpfeDemoActivity.kt
 */
//=====以下demo中关键代码

```

//1. 查询是否支持,09类型数据中包含

```
if(protoBufSupportInfo.isSupport_yylpfe()){
    integers.add(YYPFE_DATA);
}
```

//2.获取index table,这里为 YYLPFE_DATA

```
byte[] indexTab =
ProtoBufSendBluetoothCmdImpl.getInstance().itHisData(typeArray.get(currentType));

BackgroundThreadManager.getInstance().addWriteData(BleApplication.getInstance(),
indexTab);
```

// 3.1 获取具体的历史数据

```
byte[] hisData = ProtoBufSendBluetoothCmdImpl.getInstance().startHisData(type,
startSeq, endSeq);
BackgroundThreadManager.getInstance().addWriteData(context, hisData);
```

// 3.2 解析具体数据(ProtoBufDataParsePersenter.class)

```
if(hisDataType == TYPE_HIS_YYPFE){
    fixedThreadPool.execute(new Runnable() {
        @Override
        public void run() {
            mHandler.removeCallbacks(sync80Timeout);
            ProtobufYyLpfeData lpfeData =
JsonTool.fromJson(data,ProtobufYyLpfeData.class);
            //保存数据
            PbTolVHandler.saveYYLpfeToDb(lpfeData,getDataFrom());
            int progress = ProtoBufSync.getInstance().
                currentProgress(ProtoBufSync.YYPFE_DATA,lpfeData.getSeq());
            if(progress>=100){
                //同步完一段数据，发送停止同步数据，删除手表里的中医同步提醒
                //writeStopYYLpfeCmd这条指令测试阶段为了重复获取可以不下发，下发后手表会
                删除数据

                ProtoBufSync.getInstance().writeStopYYLpfeCmd();
                ProtoBufSync.getInstance().finishOneIndexTable();
            }
            mHandler.postDelayed(sync80Timeout, 5000);
        }
    });
}

//4. 发送停止同步指令(3.2代码中包含了)
public void writeStopYYLpfeCmd(){
    int type = HisDataOuterClass.HisDataType.YYPFE_DATA_VALUE;
    byte[] bytes = ProtoBufSendBluetoothCmdImpl.getInstance().stopHisData(type);

    BackgroundThreadManager.getInstance().addWriteData(BleApplication.getInstance(),
bytes);
```

```
}
```

- 发送中医建模报告 和 接收报告生成

```
/**
 * 建模，五气报告，体质报告均可以单独发送
 * 如果需要同时发送时，可调用setYYLpfeAllReport()方法
 */

/**
 * 发送五气报告
 * @param fiveQiArray 五气 字节数组
 * @param scoreArray 分数 字节数组
 */
byte[] setYYLpfeFiveQiReport(byte[] fiveQiArray,byte[] scoreArray){}

/**
 * 发送体质报告
 */
byte[] setYYLpfePhysicalReport(byte[] nameArray,byte[] scoreArray,byte[]
preNameArray,byte[] preScoreArray){}

/**
 * 发送是否建模
 */
byte[] setYYLpfeModeling(boolean isModeling){}

/**
 * 发送中医报告
 * @return 参数类如下
 */
public byte[] setYYLpfeAllReport(YYLpfeReportAllData allData){}

//同时发送建模，报告时的参数类
public class YYLpfeReportAllData {
    //是否建模
    private boolean modeling;
    //五气报告，为null时不发
    private FiveQiReportData fiveQiReport;
    //体质报告，为null时不发
    private PhysicalReportData physicalReport;
}
```



```

/**
 * 当手表有一份报告生成时，如果手表处于连接状态，则会主动上报一条数据，app可去同步数据
 * 在接收数据中 处理 13协议类型数据即可。
 */
case 0x0013:{
    ProtobufYyLpfeReportData reportData =
    JsonTool.fromJson(data,ProtobufYyLpfeReportData.class);
    //0-自动生成了一份报告，1-手动生成一份报告
    int reportType = reportData.getReportType();
}

```

其他指令

■ AGPS升级

仅仅支持GPS功能的设备支持AGPS升级,不包含的无需升级。

```

/**
 * 文件下载init
 *
 * @param fuType
 * @param fileSize
 * @param fileCyc
 * @param fileName
 * @param fileOffset
 */
byte[] setFileDescUpdate(boolean isDesc)
/**
 * 文件下载data
 *
 * @param fuType
 * @param
 * @param
 * @param buf
 */
byte[] setFileDataUpdate(int fuType, int fileCyc, int crc32AtOffset, ByteString buf)
byte[] setFileDataExit(int fuType)

```

SDK提供AGPS升级的指令.具体的功能可以看示例代码中的demo,如下:

```
// ProtoBufUpdate.Type.TYPE_GPS FONT
```

```
ProtoBufUpdate.getInstance().startUpdate(ProtoBufUpdate.Type.TYPE_GPS);
```

■ 固件升级

- 参考示例代码 `ProtoBufFirmwareUpdateActivity.class` 类,该类是固件升级的整个实现过程,可以在示例代码APP中查看固件升级的效果.添加自己的工程则需要示例代码中`no.nordicsemi.android`下面的所有文件.

■ 炬芯固件升级

1. 导入 JXblueota-release.aar 文件

2.参考示例代码JXFirmwareUpgradeActivity.kt类, 主要流程如下

```
/**
 * 1. 人工确认当前手表是否支持蓝牙3.0, 不支持则用4.0升级, 默认4.0升级
 * 2. 初始化参数 mBluzConnector: IBluzDevice 和 mOTAManager: OTAManager
 * 3. 确认当前手表是否已经被连接,
 * 连接状态下可直接调用mBluzConnector?.connect()连接设备进行升级, 实现
IBluzDevice.OnConnectionListener
 * 未连接下则需要调用 mBluzConnector?.startDiscovery(),搜索到设备后再连接设备进行升级,实现
IBluzDevice.OnDiscoveryListener
 * 4. 开始升级, 需要实现 OTAManager.OTAListener, 监听进度和升级状态
 * 5.升级成功, 释放资源
 */

//初始化参数代码:
if (mBluzConnector == null) {
    if (upgradeType == ConnectType.CLASSIC_TYPE) {
        BluzDeviceFactory.setUUID(UUID.fromString("00006666-0000-1000-8000-00805F9B34FB"))
        mBluzConnector = BluzDeviceFactory.getDevice(this,
BluzDeviceFactory.ConnectionType.SPP_ONLY)
    } else {
        mBluzConnector = BluzDeviceFactory.getDevice(this,
BluzDeviceFactory.ConnectionType.BLE)
    }
}
if (mOTAManager == null) {
    mOTAManager = OTAManager(this, mBluzConnector!!.io)
    mOTAManager!!.setListener(this)
}

//连接成功后的主要代码:
//真正的准备升级
```

```

        mOTAManager?.let {
            it.setOTAFile(upFilePath)
            it.prepare()
        }
//状态监听中开始写入文件和结束文件
        override fun onStatus(state: Int) {
            Log.v("JXUpgrade", "OTAManager Status: $state" )
            if (state == OTAManager.STATE_PREPARED) {
                mOTAStatus = OTAManager.STATE_PREPARED
                //写入文件
                mOTAManager?.upgrade()
            } else if (state == OTAManager.STATE_TRANSFERRED) {
                mOTAStatus = OTAManager.STATE_TRANSFERRED
                //升级成功
                runOnUiThread {
                    showSuccessOrFailView(true)
                }
            }
        }
    }

//进度回调
    override fun onProgress(progress: Int, total: Int) {
        Log.d("JXUpgrade", "升级的进度: onProgress: $progress - $total" )
        val mP = progress*100/total
    }

//升级成功后释放资源
    private fun releaseJx(){
        mBluzConnector?.let {
            if(mConnectDevice!=null) {
                //要保证调用了断开连接，否则会重复升级
                it.disconnect(mConnectDevice)
            }
            mOTAManager?.cancel()
            mOTAManager?.release()
            it.release()
        }
    }
}

```

接收数据

- 如接入说明步骤4中,可注册一个全局广播接收器继承 BluetoothCallbackReceiver 可全局收发广播.分到到具体的类可以使用**EventBus**来发送,实现类如下介绍:

```
/*
 *
 * @param ble_sdk_type 指SDK的类型一般来说指定的是
 Constants.Bluetooth.Zeroner_protobuf_Sdk
 * @param dataType 指令类型,可参考协议文档
 * @param data SDK解析的数据,通过JSON来解析,SDK使用的是fastJson.关于boolean类型的数据使用
 Gson有可能出现永远为false的情况,这种情况建议使用fastJson
 */
void onDataArrived(Context context, int ble_sdk_type, int dataType, String data)
```

- 全局广播一般用使用在同步历史数据解析实时数据等.针对某个单独的页面也可以使用局部广播.拷贝Demo中 BleReceiverHelper.class 到工程中的代码如下:

```
BleReceiverHelper.registerBleReceiver(context,new MyDataReceive());

private class MyDataReceive extends BluetoothCallbackReceiver{
    @Override
    public void onDataArrived(Context context, int ble_sdk_type, int dataType,
String data) {
        super.onDataArrived(context, ble_sdk_type, dataType, data);
    }
}
```

其他功能

- 设置日志输出,使用BluetoothUtils.setBluetoothLogPath和setLogEnable设置

```
/**
 * 设置蓝牙日志输出的路径 外面无需使用Environment.getExternalStorageDirectory()
 * 默认Zeroner/zeroner_3/路径
 * @param path
 */
public static void setBluetoothLogPath(String path){
    Constants.Log.LOG_DIR = path;
}
```

```
/**
 * 设置true为开启蓝牙日志输出 默认为true
 * @param enable
 */
public static void setLogEnable(boolean enable){
    Constants.Debug.LOG_FLAG = enable;
}
```